

Práctica 1. Client/Server

Miguel Ángel Dorado Maldonado

26 Septiembre

Cambio N1. Error de compilación

P1_echo_server.c: En la función 'main':

P1_echo_server.c:57:13: error: 'struct sockaddr_in' no tiene un miembro llamado 'sin_prt'; ¿quiso decir 'sin_port'?

```
57 |     serv_addr.sin_prt = htons(PORT);  
    |                   ^~~~~~  
    |                   sin_port
```

Cambio N2. Puerto privilegiado

bind: Permission denied

El puerto 80 es un puerto privilegiado para el que hacen falta permisos de administrador. Se cambia al puerto 9000 como indica el script de ejecución.

Cambio N3. Tamaño de mensaje

Para tamaño inferior a buffer:

```
server: client connected from 127.0.0.1:53904  
server: received 1000 bytes  
server: client disconnected
```

Para tamaño superior a buffer (p.ej. 10000):

```
server: client connected from 127.0.0.1:40196  
server: received 1024 bytes  
server: client disconnected
```

Observaciones

Cuando el tamaño de mensaje es mayor al buffer de lectura, el mensaje se corta ya que solo se hace una lectura por conexión.

Para solucionarlo, meter la lectura en un bucle y controlar la cantidad de bytes recibidos cada vez.

STAGE 1. Ejecución correcta

Salida de ejecución exitosa con cliente mandando 5000 bytes:

SERVIDOR:

```
server: listening on port 9000
server: client connected from 127.0.0.1:38224
server: received 1024 bytes
server: received 1024 bytes
server: received 1024 bytes
server: received 1024 bytes
server: received 904 bytes
server: client disconnected
```

CLIENTE:

```
client: 5000 bytes echoed in 0.075 ms (identical)
```

STAGE 2. Tabla de velocidades

Size	Average Time (ms)	Chunks needed
10	0,0332	1
1000	0,0338	1
10000	0,1206	10
100000	40,7604	98

El tiempo no crece proporcionalmente al tamaño de mensaje.

Todas las peticiones, independientemente del tamaño, tienen un tiempo “fijo” que no depende de los datos enviados. Apertura de conexión, llamadas al sistema, etc...

Con valores bajos el tiempo es similar. He probado también con 1M de bytes y el tiempo medio es similar a 100000. Parece que gran parte del tiempo es por conexión o retardos del buffer más que por la cantidad de datos enviados.

STAGE 3.

Parar servidor

Como los tiempos son muy cortos, he añadido temporalmente un sleep antes de hacer los writes al cliente.

SERVIDOR:

```
server: listening on port 9000
server: client connected from 127.0.0.1:37068
server: received 1024 bytes
```

^C

CLIENTE:

```
read: Connection reset by peer
```

Al cerrar el servidor con ctrl+c, el proceso se termina y se cierra el socket. El cliente recibe un reset por lo que read() devuelve -1 entrando en la rama:

```
if (n < 0) {  
    perror("read");  
    return 1;  
}
```

El error aparece de inmediato.

Conectar cliente a servidor no existente

CLIENTE:

```
connect: Connection refused
```

El cliente no encuentra ningún socket en ese puerto al que conectarse y devuelve un error de forma inmediata.

Conectar dos clientes al servidor

El servidor no maneja los clientes de forma asíncrona. Debe cerrar la conexión con uno para atender al otro.

Dejo un sleep temporal de 2 segundos para poder lanzar ambos clientes antes de que el primero termine exitosamente.

Arranco el servidor, el primer cliente se conecta y empieza a intercambiar mensajes. El segundo cliente se conecta y se queda esperando a que el servidor vuelva a llamar a `accept()`.

Cuando termina la primera conexión, el servidor acepta la conexión pendiente y comienza el intercambio.

TRAZA:

```
server: listening on port 9000  
server: client connected from 127.0.0.1:47416  
server: received 1024 bytes  
server: received 1024 bytes  
server: received 952 bytes  
server: client disconnected  
client: 3000 bytes echoed in 6000.279 ms (identical)  
server: client connected from 127.0.0.1:47430  
server: received 1024 bytes  
server: received 1024 bytes  
server: received 952 bytes  
server: client disconnected  
client: 3000 bytes echoed in 11386.064 ms (identical)
```

Extra. Servidor asíncrono

He ampliado la práctica añadiendo soporte para múltiples conexiones simultáneas.

Por cada conexión se crea un hijo, se modifican los file descriptors correspondientes para no bloquear el proceso padre, se maneja la comunicación en el hijo y se limpia el proceso cuando la conexión termina.

Dejo a continuación el código del servidor modificado.

C/C++

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#define PORT 9000
#define BUFSIZE 1024

void clean_child(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0) {
    }
}

int main(void) {
    int listen_fd, conn_fd;
    struct sockaddr_in serv_addr, cli_addr;
    socklen_t cli_len;
    char buffer[BUFSIZE];
    ssize_t n;

    signal(SIGCHLD, clean_child);

    listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (listen_fd < 0) {
        perror("socket");
        exit(1);
    }
```

```

{
    int yes = 1;
    if (setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &yes,
sizeof(yes)) <
        0) {
        perror("setsockopt");
        exit(1);
    }
}

memset(&serv_addr, 0, sizeof(serv_addr));
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);

serv_addr.sin_port = htons(PORT);

if (bind(listen_fd, (struct sockaddr *)&serv_addr,
sizeof(serv_addr)) < 0) {
    perror("bind");
    exit(1);
}

if (listen(listen_fd, 8) < 0) {
    perror("listen");
    exit(1);
}

printf("server: listening on port %d\n", PORT);
fflush(stdout);

for (;;) {
    cli_len = sizeof(cli_addr);
    conn_fd = accept(listen_fd, (struct sockaddr *)&cli_addr,
&cli_len);
    if (conn_fd < 0) {

```

```

    perror("accept");
    continue;
}

printf("server: client connected from %s:%d\n",
       inet_ntoa(cli_addr.sin_addr),
       ntohs(cli_addr.sin_port));
fflush(stdout);

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    close(conn_fd);
    continue;
}

if (pid == 0) {
    close(listen_fd);

    while ((n = read(conn_fd, buffer, BUFSIZE)) > 0) {
        printf("server: received %zd bytes\n", n);
        fflush(stdout);

        ssize_t total = 0;
        sleep(2);

        while (total < n) {
            ssize_t current = write(conn_fd, buffer + total, n -
total);
            if (current <= 0) {
                if (current < 0) {
                    perror("write");
                }
                break;
            }
            total += current;
        }
    }
}

```

```
    if (n < 0) {  
        perror("read");  
    }  
  
    printf("server: client disconnected\n\n");  
    fflush(stdout);  
    close(conn_fd);  
    exit(0);  
  
} else {  
    close(conn_fd);  
}  
}  
  
return 0;  
}
```